

Scientific Computing: Exercises

Thorsten Koch



Applied Algorithmic Intelligence Methods department at ZIB
Chair of *Software and Algorithms for Discrete Optimization*
at TU Berlin



The solution must be submitted by the following Thursday before the lecture.

Programming exercises must be submitted as follows:

- ▷ Put everything together in one .zip file. **This should be named:**
 - **exNN-MMMMMM-lastname-firstname.zip**
 - NN = 01 .. 17 number of the exercise
 - MMMMMM is your 6-digit Matrikelnummer (all who have none, send a message on ISIS and we assign you a number)
 - lastname is your Lastname (only use characters a-z in English alphabet)
 - firstname is your Firstname (only use characters a-z in English alphabet)
- ▷ The .zip file should extract to a folder of the same name (without the .zip extension).
- ▷ When we do cargo run ... in that folder, your program must produce the desired output.
- ▷ The programs should include comments to explain what they do,
- ▷ NO HARD CODED PATHS, no warnings from cargo clippy.
- ▷ In written exercises, the steps for solving the problem need to be explained appropriately.
- ▷ **We will ignore any submission that does not conform to the above.**
- ▷ **All exercises must be written in English.**

We will put up an PDF with the latest exercise on ISIS every Thursday after the lecture.

Written exercises have to be submitted the same way, only as an .pdf file instead of a .zip file.

There will be at least 12 programming exercises, giving 2 points each, and 12 theory exercises giving 1 point each. The maximum number of points that will count for the final grade, however, is 30.

Let S be the set of all students with timely submissions.

Let $C \subseteq S$ be the subset of students with *acceptable* submissions.

An *acceptable* submission is one where the result looks ok.

Let T be the set of students in attendance at the Tuesday Exercise session.

Let $X = C \cap T$ be the students valid for presentation.

Each student in X gets tentatively points credited, accumulating from week to week.

We will pick samples $x_1, x_2, \dots, x_n \in X$ to present their results each week.

Your performance at the presentation is rated with a factor $p \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$.

The accumulated tentatively credited points are multiplied by p , rounded up, and become permanent.

At the end of the lecture all points credited at that time become permanent automatically.

L02

Numbers

1. Given 8 binary inputs $x_0, \dots, x_7$ and 4 binary outputs $s_0, \dots, s_3$, design a circuit where the output is the binary representation of the count of the 1 values in the input, i.e.

$$\sum_{i=0}^7 x_i = \sum_{i=0}^3 2^i s_i$$

x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	s_3	s_2	s_1	s_0
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	0	0	0	0	1
0	1	0	0	0	1	0	0	0	0	1	0
0	0	1	1	1	0	0	0	0	0	1	1
...	...	...	...	...	...	...	...	...	...	...	...
1	0	1	0	1	0	1	0	0	1	0	0
1	1	1	0	0	1	0	1	0	1	0	1
0	1	1	1	1	1	1	1	0	1	1	1
1	1	1	1	1	1	1	1	1	0	0	0

Example of some inputs and output.
The full table would have 256 entries.

2. Let `x: i32`, when is `(x & (x - 1)) == 0` true and why?
3. The following are the binary values of some single precision (32 bit) IEEE754 floating point values:

3FCF 1BBD
FF80 0000
8800 0000
3E4C CCCD
3FDD B3D7

Which numbers do they represent exactly in decimal and which real number are they supposed to represent?

Example: `C000 0000` = exact decimal -2.0 real -2.0

```
1  /// Demonstration of 4-bit gate level binary addition and multi
2  /// [See also](https://coertvonk.com/inquiries/computer-math/di
3  ///
4  /// Scientific Computing
5  /// TK 040ct2024–200ct2024
6  ///
7  use std::env;
8
9  /// 2-bit add with carry
10 ///
11 /// | a | b | ci | so | co |
12 /// |---|---|---|---|---|
13 /// | 0 | 0 | 0 | 0 | 0 |
14 /// | 0 | 0 | 1 | 1 | 0 |
15 /// | 0 | 1 | 0 | 1 | 0 |
16 /// | 1 | 0 | 0 | 1 | 0 |
17 /// | 0 | 1 | 1 | 0 | 1 |
18 /// | 1 | 0 | 1 | 0 | 1 |
19 /// | 1 | 1 | 0 | 0 | 1 |
20 /// | 1 | 1 | 1 | 1 | 1 |
21 /// |---|---|---|---|---|
22 fn fa(a: bool, b: bool, ci: bool) -> (bool, bool) {
23   let so = a ^ b ^ ci;
24   let co = a && b || ci && (a ^ b);
25
26   (so, co)
27 }
28
29 /// 2-bit multiply with cary
30 ///
31 /// | x | y | si | ci | b | co | so |
32 /// |---|---|---|---|---|---|
33 /// | 0 | * | 0 | 0 | 0 | 0 | 0 |
34 /// | * | 0 | 0 | 0 | 0 | 0 | 0 |
35 /// | 0 | * | 0 | 1 | 0 | 0 | 1 |
36 /// | * | 0 | 0 | 1 | 0 | 0 | 1 |
37 /// | 0 | * | 1 | 0 | 0 | 0 | 1 |
38 /// | * | 0 | 1 | 0 | 0 | 0 | 1 |
39 /// | 0 | * | 1 | 1 | 0 | 1 | 0 |
40 /// | * | 0 | 1 | 1 | 0 | 1 | 0 |
41 /// | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
42 /// | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
43 /// | 1 | 1 | 1 | * | 1 | 1 | 0 |
44 /// |---|---|---|---|---|---|
45 fn ma(x: bool, y: bool, ci: bool, si: bool) -> (bool, bool) {
46   fa(si, x && y, ci)
47 }
48
```

```
49  /// add two times 4 bits with carry
50  #[allow(clippy::too_many_arguments)]
51  fn add4(
52     a0: bool, a1: bool, a2: bool, a3: bool,
53     b0: bool, b1: bool, b2: bool, b3: bool)
54  -> (bool, bool, bool, bool, bool) {
55     let (s0, c0) = fa(a0, b0, false);
56     let (s1, c1) = fa(a1, b1, c0);
57     let (s2, c2) = fa(a2, b2, c1);
58     let (s3, c3) = fa(a3, b3, c2);
59
60     (s0, s1, s2, s3, c3)
61  }
62
63  /// multiply two times 4 bits giving 8 bits
64  #[allow(clippy::too_many_arguments)]
65  fn mul4x4(
66     x0: bool, x1: bool, x2: bool, x3: bool,
67     y0: bool, y1: bool, y2: bool, y3: bool)
68  -> (bool, bool, bool, bool, bool, bool, bool, bool) {
69
70     // Fill in the missing code
71  }
```

Exercise 02 – Programming part 1

```

73  /// Split an integer between 0 and 15 into 4 bits
74  fn split4(x: u8) -> (bool, bool, bool, bool) {
75      assert!(x < 16);
76
77      (x & 1 != 0, x & 2 != 0, x & 4 != 0, x & 8 != 0)
78  }
79
80  /// Merge 4 bits giving an integer
81  fn merge4(x0 : bool, x1: bool, x2: bool, x3: bool) -> u8 {
82      ( if x0 { 1 } else { 0 })
83      + (if x1 { 2 } else { 0 })
84      + (if x2 { 4 } else { 0 })
85      + (if x3 { 8 } else { 0 })
86  }
87
88  /// Merge 8 bits giving an integer
89  #allow(clippy::too_many_arguments)]
90  fn merge8(x : (bool, bool, bool, bool, bool, bool, bool, bool)) -> u8 {
91      merge4(x.0, x.1, x.2, x.3) + 16 * merge4(x.4, x.5, x.6, x.7)
92  }

```

Above are some utility functions to split integers into bits (Booleans) and to merge them again.

On the right you see the main program, taking and checking two numbers from the command line.

```

114  /// Add and multiply two numbers between 0 and 15.
115  /// Example: cargo run 4 7
116  fn main() {
117      // parse and check command line arguments
118      let args: Vec<String> = env::args().collect();
119
120      if args.len() < 3 {
121          eprintln!("usage: {} number number", &args[0]);
122          std::process::exit(1);
123      }
124      let x = args[1].parse().unwrap_or_else(|err| panic!("argument 1 \"{}\": {err}", &args[1]));
125      let y = args[2].parse().unwrap_or_else(|err| panic!("argument 2 \"{}\": {err}", &args[2]));
126
127      if x > 15 || y > 15 {
128          panic!("Argument > 15: {x} {y}");
129      }
130
131      // Split the number into bits
132      let a = split4(x);
133      let b = split4(y);
134
135      // Add the bits and print the result
136      let z = add4(a.0, a.1, a.2, a.3, b.0, b.1, b.2, b.3);
137
138      print!("{}", x + y);
139
140      if z.4 {
141          println!("Overflow!");
142      } else {
143          let r = merge4(z.0, z.1, z.2, z.3);
144          assert_eq!(x + y, r);
145          println!("{}", r);
146      }
147
148      // Multiply the bits and print the result
149      let z = mul4x4(a.0, a.1, a.2, a.3, b.0, b.1, b.2, b.3);
150      let r = merge8(z);
151      assert_eq!(x * y, r);
152      println!("{}", x * y);
153  }
154

```


On the right are test routines, that check every possible allowed value for the addition and multiplication.

You can find the program on ISIS.

First use cargo to create a new crate called “gate”.
Then copy the file main.rs into the gate/src folder.

You can run

```
$ cargo build
$ cargo run 4 8
$ cargo test
$ cargo clippy
$ cargo doc --open
```

```
155 /// Check addition for all allowed input values
156 #[test]
157 fn check_add() {
158     for x in 0 ..= 15 {
159         for y in 0 ..= 15 {
160             let a = split4(x);
161             let b = split4(y);
162             let z = add4(a.0, a.1, a.2, a.3, b.0, b.1, b.2, b.3);
163             if x + y > 15 {
164                 assert_eq!(z.4, true);
165             } else {
166                 assert_eq!(z.4, false);
167                 let r = merge4(z.0, z.1, z.2, z.3);
168                 assert_eq!(x + y, r);
169             }
170         }
171     }
172 }
173
174 /// Check multiplication for all allowed values
175 #[test]
176 fn check_mul() {
177     for x in 0 ..= 15 {
178         for y in 0 ..= 15 {
179             let a = split4(x);
180             let b = split4(y);
181             let z = mul4x4(a.0, a.1, a.2, a.3, b.0, b.1, b.2, b.3);
182             let r = merge8(z);
183             assert_eq!(x * y, r);
184         }
185     }
186 }
```

The Collatz sequence is defined here:

https://en.wikipedia.org/wiki/Collatz_conjecture#Statement_of_the_problem

(See also Hugardy, Vygen, page 18)

The numbers a_i in the sequence are also called *Hailstone* numbers.

Write a rust program to compute the smallest starting value that leads to a hailstone number a_i being bigger than 2^{64} . Output the starting value, the first reach value $> 2^{64}$, maximum value and the total sequence length to reach 1 for this starting value.

The result must be printed to the console, e.g.,

```
$ cargo run
```

```
2117 12636374748498 2536474839390404 566363
```